Aula 06 – Aprendizado por Reforço e Q-Learning

Simulações Computacionais para o Ensino de Robótica Inteligente e Aprendizado por Reforço

Leonardo de Souza Gomes
Orientador: Prof. Dr. André Luiz C. Ottoni

Edital PROGRAD nº 27, de 26 de maio de 2026
Universidade Federal de Ouro Preto

2026

Objetivo: compreender os fundamentos do algoritmo Q-Learning e sua aplicação em robótica para navegação autônoma.

Ao final desta aula, você deverá entender:

- o que é aprendizado por reforço e onde o Q-Learning se encaixa;
- os conceitos de estado, ação, recompensa e política;
- a equação de atualização do Q-Learning;
- o papel dos hiperparâmetros α, γ e ϵ;
- como o Q-Learning é implementado no controle de um robô no Webots.

Introdução: Aprender por Tentativa e Erro

Imagine ensinar o robô e-puck a desviar de obstáculos. Você sabe o **objetivo** (não colidir), mas não sabe dizer, para cada leitura dos sensores de distância, qual é o comando *exato* dos motores.

O problema

Como o robô pode aprender a se comportar bem sem ninguém indicar, a cada situação, qual é a ação correta?

A saída: aprender com as consequências

O robô **age**, observa o que acontece e recebe um sinal indicando se algo bom (avançar sem bater) ou ruim (colidir) ocorreu. Com a experiência acumulada, ele passa a preferir as ações que levam a bons resultados.

Esse é o princípio do **aprendizado por reforço** [1]: aprender o que fazer a partir da interação com o ambiente, e não a partir de exemplos prontos.

Recompensa e Reforço

Esse sinal de sucesso ou fracasso é chamado de **recompensa**, ou **reforço** [1].

Frequência varia com a tarefa (e-puck)

- **Navegar até um destino**: reforço só ao chegar ao objetivo;
- **Seguir uma linha**: cada leitura sobre a linha é uma recompensa;
- **Desviar de obstáculos**: cada passo à frente já é um progresso, e cada colisão, uma penalidade.

Da mesma forma que animais são "programados" para reconhecer dor e fome como recompensas negativas, e prazer e alimento como positivas, um agente de RL precisa ser projetado para identificar essa parte da percepção como recompensa, e não apenas como mais uma entrada sensorial.

Definição da tarefa

Aprendizado por Reforço é usar recompensas **observadas** para aprender uma política ótima (ou quase ótima) para o ambiente.

Conceitos Fundamentais

Estado (s)

Configuração atual do agente no ambiente.

Ex.: sensores indicam obstáculo à frente.

Recompensa (r)

Feedback imediato após a ação.

Ex.: $+1$ por avançar, -10 por colisão.

Ação (a)

Movimento que o agente pode executar.

Ex.: seguir em frente, virar.

Política (π)

Regra que mapeia estados para ações.

Ex.: escolher a ação de maior valor.

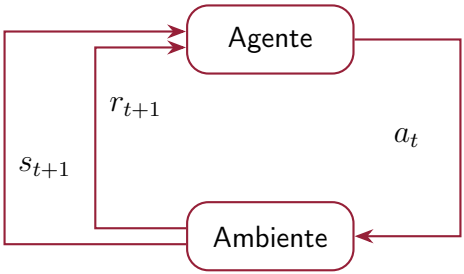
O ciclo de interação

O agente observa s, escolhe a; o ambiente responde com r e um novo estado s'. O ciclo se repete, e o agente aprende com o que observa.

Ciclo do aprendizado por reforço

O objetivo primário do agente é **maximizar o acúmulo de recompensas** no longo prazo, ajustando suas ações com base no feedback recebido do ambiente a cada passo de tempo t.

- **Ação (a_t):** Decisão tomada pelo agente;
- **Recompensa (r_{t+1}):** Feedback numérico imediato;
- **Estado (s_{t+1}):** Nova situação do ambiente.

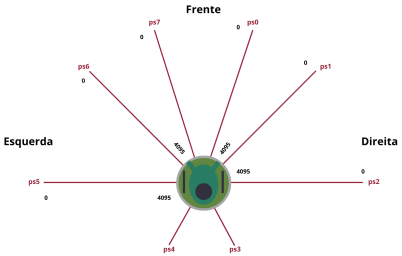


O que é Q-Learning?

Q-Learning é um algoritmo de aprendizado por reforço que aprende uma função de valor $Q(s, a)$ representando a **qualidade** de cada ação em cada estado. A letra Q significa Qualidade (*Quality*).

Exemplo: Robô no Webots

- **Estado**: leitura dos sensores de distância;
- **Ações**: frente, virar à esquerda, virar à direita;
- **Recompensa**: $+1$ por avançar, -10 por colisão;
- $Q(s, a)$: o quanto vale, a longo prazo, tomar a ação a no estado s.



Objetivo

Encontrar a melhor sequência de ações para maximizar recompensas futuras a partir do estado atual [2].

Características do Q-Learning

Sem modelo (*model-free*)

O robô **não precisa saber** como o ambiente funciona (física, posição das paredes, o que cada sensor lerá depois de uma ação). Ele aprende apenas **experimentando**: age, observa r e s' e atualiza Q.

Baseado em valor (*value-based*)

O algoritmo aprende **valores** $Q(s, a)$, e a política surge deles: em cada estado, escolher a ação de maior Q ($\pi(s) = \arg\max_a Q(s, a)$). Não há uma regra de comportamento escrita à mão.

Off-policy

Significa que o algoritmo aprende sobre uma política diferente da que usa para agir.

- **Para agir**: ϵ-greedy (às vezes explora ao acaso);
- **Para aprender**: a gulosa, pois o alvo usa $\max_{a'} Q(s', a')$.

A Q-Table

A Q-table armazena um valor $Q(s, a)$ para cada combinação de estado e ação. No nosso caso: 8 estados (sensores) $\times$ 3 ações (movimentos) = 24 valores.

Estado	Frente (0)	Esquerda (1)	Direita (2)
0 – livre	0.82	0.10	0.12
1 – obstáculo frente	-0.30	0.45	0.51
2 – obstáculo direita	0.60	0.20	-0.15
3 – frente + direita	-0.10	0.55	0.05
4 – obstáculo esquerda	0.58	-0.20	0.18
5 – frente + esquerda	-0.05	0.08	0.49
6 – esquerda + direita	0.40	-0.22	-0.18
7 – cercado (todas as zonas)	-0.40	0.12	0.09

Em cada linha, a célula destacada é a ação com **maior** $Q(s, a)$, ou seja, a ação que a política gulosa ($\arg\max_a Q(s, a)$) escolheria para aquele estado.

A Equação do Q-Learning

A atualização é uma regra de **diferença temporal**, derivada da equação de Bellman:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big]$$

- s, a: estado atual e ação executada;
- r: recompensa recebida;
- s': novo estado;
- a': qualquer ação possível em s';
- α: taxa de aprendizado;
- γ: fator de desconto.

O termo $\gamma \max_{a'} Q(s',a')$

- $\max_{a'} Q(s',a')$: o **melhor valor** que o robô conhece a partir do novo estado s';
- γ: **desconta** esse valor, pois recompensas futuras valem menos que as imediatas.

O alvo soma o que ganhei **agora** (r) com o que espero ganhar **depois** ($\gamma \max_{a'} Q(s',a')$). $Q(s,a)$ é ajustado na direção desse alvo, com passo α.

Exemplo de Atualização de Q

Suponha:

- Estado atual: obstáculo à frente ($s = 1$);
- Ação: virar à direita ($a = 2$);
- Recompensa: $r = -0.1$ (girar sem colidir);
- Novo estado: livre ($s' = 0$);
- $\max_{a'} Q(s', a') = 0.5$;
- $Q(s, a) = 0.0$ (inicial).

Atualização:

$$\begin{aligned} Q(1,2) &\leftarrow 0.0 + 0.2 \times [(-0.1) + 0.9 \times 0.5 - 0.0] \\ &= 0.0 + 0.2 \times [0.35] = 0.07 \end{aligned}$$

A ação "virar à direita" nesse estado ganha valor positivo.

Hiperparâmetros: ϵ (Exploração)

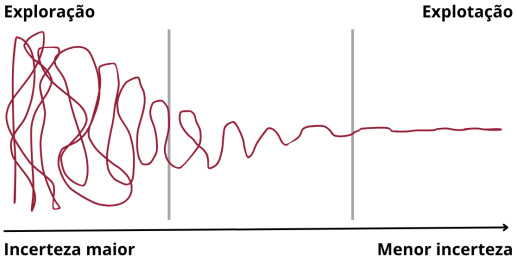
ϵ (Epsilon) controla o **equilíbrio** entre exploração e explotação.

Epsilon-Greedy

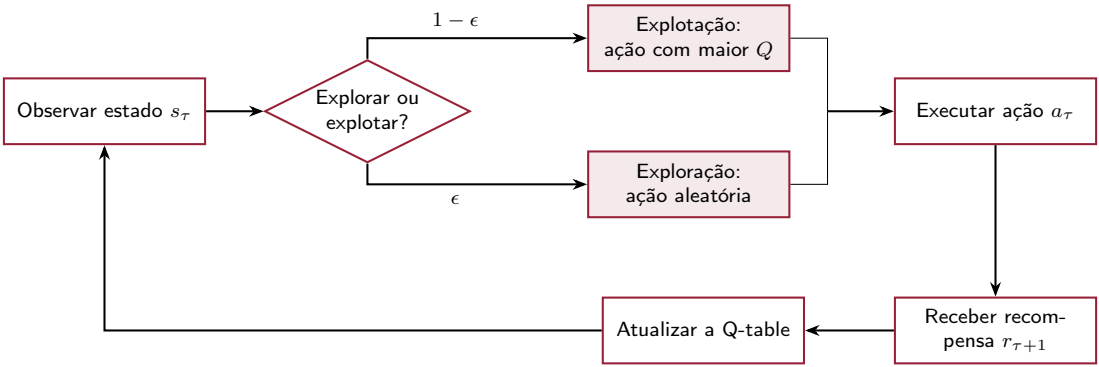
- Com probabilidade ϵ: ação aleatória (explorar);
- Com probabilidade $1 - \epsilon$: melhor ação conhecida (explotar).

Decaimento de ϵ

- $\epsilon_{inicial} = 0.3$: exploração inicial;
- $\epsilon_{min} = 0.05$: exploração mínima;
- $\epsilon_{decay} = 0.999$: decaimento lento.



Ciclo do aprendizado por reforço



Leitura do diagrama

A cada passo, o agente sorteia: com probabilidade ϵ explora (ação aleatória) e com $1-\epsilon$ explota (ação de maior Q). Depois, recebe a recompensa e atualiza a Q-table.

Algoritmo Q-Learning

Inicializar $Q(s, a) = 0$ para todos s, a
Definir $\epsilon = \epsilon_{inicial}$
repeat
    Observar estado atual s
    Escolher ação a usando ϵ-greedy
    Executar ação a, observar r e novo estado s'
    $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
    $s \leftarrow s'$
    ϵ: $\epsilon \leftarrow \epsilon \times \epsilon_{decay}$
until término da simulação

Convergência

Com exploração suficiente e α adequado, o algoritmo converge para a política ótima [2].

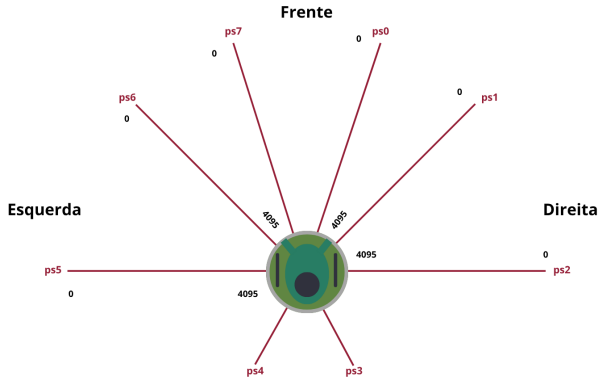
Estados do robô

O robô possui 8 sensores de distância infravermelhos, agrupados em 3 zonas: frente, esquerda e direita. Cada sensor retorna um valor entre 0 e 4095, que cresce conforme o obstáculo se aproxima: 0 indica caminho livre e 4095 indica obstáculo muito próximo.

Estados ($2^3 = 8$)

Cada zona pode ter obstáculo ou não:

- Frente: sensores ps0 e ps7;
- Direita: sensores ps1 e ps2;
- Esquerda: sensores ps5 e ps6.



Por que Discretizar os Estados?

O que é discretizar?

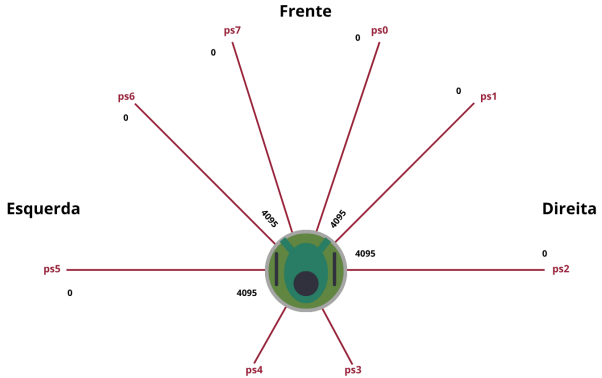
Transformar valores contínuos (ou muito numerosos) em poucas categorias.

O problema

Cada sensor tem 4096 valores possíveis. Com 8 sensores seriam $4096^8 \approx 8 \times 10^{28}$ combinações, impossível de guardar numa Q-table.

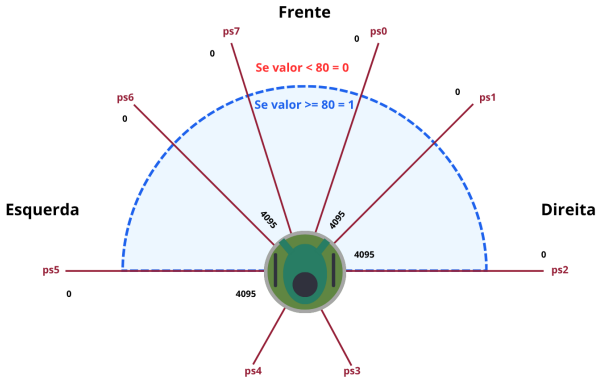
O custo

Perdemos detalhe: leituras diferentes acima do limiar viram o mesmo estado. Em troca, a Q-table passa a ter apenas $8 \times 3 = 24$ valores.



Do Sensor ao Estado

1. Em cada zona, comparar as leituras com o `LIMIAR`, neste caso 80: igual ou acima dele é obstáculo (1), senão é livre (0);
2. Combinar os 3 bits em um número de 0 a 7.



No código: Parâmetros e Q-table

Todos os hiperparâmetros e as dimensões da tabela ficam no topo do arquivo:

```
#define NUM_ESTADOS 8       // 2^3 combinacoes (frente, esquerda, direita)
#define NUM_ACOES   3       // 0 = frente, 1 = esquerda, 2 = direita

#define ALPHA 0.2           // taxa de aprendizado
#define GAMMA 0.9           // fator de desconto
#define EPSILON_INICIAL 0.3
#define EPSILON_MIN 0.05
#define EPSILON_DECAY 0.999

double Q[NUM_ESTADOS][NUM_ACOES];   // a Q-table
```

Leitura

A Q-table é apenas uma matriz 8×3: a linha é o estado e a coluna é a ação.

Função calcular_estado

```
#define LIMIAR_OBSTACULO 80.0
#define LIMIAR_COLISAO   150.0
int calcular_estado(double ps_values[8], bool *colidiu) {
  bool frente   = ps_values[0] >= LIMIAR_OBSTACULO || ps_values[7] >=
    LIMIAR_OBSTACULO;
  bool direita  = ps_values[1] >= LIMIAR_OBSTACULO || ps_values[2] >=
    LIMIAR_OBSTACULO;
  bool esquerda = ps_values[5] >= LIMIAR_OBSTACULO || ps_values[6] >=
    LIMIAR_OBSTACULO;
  *colidiu = false;
  for (int i = 0; i < 8; i++) {
    if (ps_values[i] > LIMIAR_COLISAO) { *colidiu = true; break; }
  }
  return (frente ? 1 : 0) + (direita ? 2 : 0) + (esquerda ? 4 : 0);
}
```

Converte as 8 leituras em um estado de 0 a 7 e, ao mesmo tempo, detecta colisão: qualquer sensor acima de 150 indica que o robô está quase encostando.

Função escolher_acao

Implementa o ϵ-greedy: sorteia $r \in [0, 1]$.
Se $r < \epsilon$,explora;
senão, escolhe $\arg\max_a Q(s, a)$.

```
int escolher_acao(int estado, double epsilon) {
  double r = (double) rand() / RAND_MAX;
  if (r < epsilon) {
    return rand() % NUM_ACOES;              // exploracao: acao aleatoria
  } else {
    int melhor_acao = 0;                    // explotacao: maior Q do estado
    for (int a = 1; a < NUM_ACOES; a++)
      if (Q[estado][a] > Q[estado][melhor_acao]) melhor_acao = a;
    return melhor_acao;
  }
}
```

Função aplicar_acao

Traduz a ação (0, 1 ou 2) em velocidades dos dois motores. Girar é fazer as rodas andarem em sentidos opostos.

```
void aplicar_acao(int acao, double *left_speed, double *right_speed) {
  switch (acao) {
    case 0:  // frente
    *left_speed = 0.5 * MAX_SPEED;  *right_speed = 0.5 * MAX_SPEED;  break;
    case 1:  // virar esquerda
    *left_speed = -0.3 * MAX_SPEED; *right_speed = 0.5 * MAX_SPEED;  break;
    case 2:  // virar direita
    *left_speed = 0.5 * MAX_SPEED;  *right_speed = -0.3 * MAX_SPEED; break;
  }
}
```

Função calcular_recompensa

Recompensas

É aqui que o projetista **define o objetivo** do robô. Mudar estes três números muda o comportamento aprendido.

- Colisão: -10 (penalidade forte);
- Avançar: $+1$ (incentivo);
- Girar: -0.1 (pequeno custo).

```
double calcular_recompensa(bool colidiu, int acao) {
  if (colidiu) return -10.0;
  if (acao == 0) return 1.0;
  return -0.1;
}
```

Função max_Q

Calcula $\max_{a'} Q(s', a')$, o termo da equação que olha para o **próximo** estado. Diferente de `escolher_acao`, ela devolve o **valor**, não a ação.

```
double max_Q(int estado) {
  double max_val = Q[estado][0];
  for (int a = 1; a < NUM_ACOES; a++)
  if (Q[estado][a] > max_val) max_val = Q[estado][a];
  return max_val;
}
```

No código: Inicialização no `main`

```
wb_robot_init();
srand(time(NULL));                        // semente do sorteio (epsilon-greedy)

for (i = 0; i < 8; i++) {                 // liga os 8 sensores
  ps[i] = wb_robot_get_device(ps_names[i]);
  wb_distance_sensor_enable(ps[i], TIME_STEP);
}
left_motor  = wb_robot_get_device("left wheel motor");
right_motor = wb_robot_get_device("right wheel motor");
wb_motor_set_position(left_motor, INFINITY);    // controle por velocidade
wb_motor_set_position(right_motor, INFINITY);

for (i = 0; i < NUM_ESTADOS; i++)         // Q-table zerada
for (int a = 0; a < NUM_ACOES; a++) Q[i][a] = 0.0;

double epsilon = EPSILON_INICIAL;
int estado_atual = calcular_estado(ps_values, &colidiu);   // estado inicial
```

Loop Principal no Webots

```c
while (wb_robot_step(TIME_STEP) != -1) {
  // 1. Escolhe e aplica a acao
  int acao = escolher_acao(estado_atual, epsilon);
  aplicar_acao(acao, &left_speed, &right_speed);
  wb_motor_set_velocity(left_motor, left_speed);
  wb_motor_set_velocity(right_motor, right_speed);
  // 2. Le sensores e calcula o novo estado
  for (i = 0; i < 8; i++) ps_values[i] = wb_distance_sensor_get_value(ps[i]);
  int novo_estado = calcular_estado(ps_values, &colidiu);
  // 3. Calcula a recompensa
  double recompensa = calcular_recompensa(colidiu, acao);
  // 4. Atualiza a Q-table
  Q[estado_atual][acao] += ALPHA * (recompensa + GAMMA * max_Q(novo_estado)
  - Q[estado_atual][acao]);
  // 5. Avanca o estado e decai epsilon
  estado_atual = novo_estado;
  if (epsilon > EPSILON_MIN) epsilon *= EPSILON_DECAY;
}
```

Resultados Esperados

Após milhares de iterações, o robô deve evitar colisões na maioria das situações e preferir ações que levam a estados "seguros".

Acompanhamento

Use logs no console para monitorar:

- Evolução de ϵ;
- Recompensas médias por intervalo;
- Valores Q para estados críticos.

```
if (passo % 100 == 0) {
  printf("Passo %d | epsilon=%.3f | estado=%d\n",
  passo, epsilon, estado_atual);
}
```

Vantagens e Limitações

Vantagens

- Simples de implementar;
- Não requer modelo do ambiente;
- Converge para política ótima (teoricamente);
- Funciona bem em espaços discretos pequenos.

Limitações

- Q-Table cresce exponencialmente com estados;
- Não lida bem com espaços contínuos;
- Pode demorar para convergir;
- Sensível a hiperparâmetros (α, γ, ϵ).

Atividade Prática

Desafio

Implemente e treine o Q-Learning no Webots para o robô e-puck navegar evitando obstáculos.

- Ajuste os hiperparâmetros (α, γ, ϵ);
- Modifique a função de recompensa;
- Teste diferentes discretizações de estado;
- Observe o comportamento antes e depois do treino.

Agradecimentos

Este trabalho foi realizado com o apoio da Pró-Reitoria de Graduação (PROGRAD), do Grupo de Pesquisa Machine Learning and Robotics (MLBots) e do Departamento de Computação (DECOM) da Universidade Federal de Ouro Preto (UFOP).

Leonardo de Souza Gomes, leonardo.sg@aluno.ufop.br

Prof. Dr. André Luiz C. Ottoni, andre.ottoni@ufop.edu.br

Edital PROGRAD nº 27, de 26 de maio de 2026







Referências

[1] S. Russell e P. Norvig, *Inteligência Artificial: Uma Abordagem Moderna*, 3ª ed. Elsevier, 2013.

[2] Abid Ali Awan, *Introdução ao Q-learning: um tutorial para iniciantes*, https://www.datacamp.com/pt/tutorial/introduction-q-learning-beginner-tutorial, Acesso em: 17 set. 2026, 2026. (acesso em 17/09/2026).